

Problem 1

Problem Statement: Interactive Feedback Button

Scenario:

A startup company wants a simple interactive webpage where users can click a button to submit feedback and instantly see a confirmation message without refreshing the page.

z'' Requirements

- Create a webpage with a “Submit Feedback” button.
- When clicked, display a confirmation message dynamically.
- Use JavaScript event listeners to handle the click event.

z\' Technical Constraints

- Use plain HTML, CSS, and JavaScript.
- Event handling must use `addEventListener()`.
- No external frameworks allowed.

Expectations:

- Proper DOM manipulation.
- Clean and readable code.

🎯 Learning Outcome

- Understand browser event handling.
- Learn DOM manipulation basics.

Source Code:

File Name: Index.html Output:

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Interactive Feedback</title>
7  <style>
8      body {
9          font-family: Arial, Helvetica, sans-serif;
10         text-align: center;
11         margin-top: 100px;
12         background-color: lightsteelblue;
13     }
14     button {
15         padding: 10px 20px;
16         font-size: 16px;
17     }
18     #message {
19         margin-top: 20px;
20         font-size: 18px;
21     }
22 </style>
23 </head>
24 <body>
25     <h2>Feedback Form</h2>
26     <textarea rows="10" cols="40" placeholder="Write your Feedback Here..."></textarea><br><br>
27     <button id="feedbackbtn">Submit Feedback</button>
28     <p id="message"></p>
29     <script>
30         const button = document.getElementById("feedbackbtn");
31         const message = document.getElementById("message");
32         const textarea = document.querySelector("textarea");
33         button.addEventListener("click", function () {
34             if (textarea.value.trim() === "") {
35                 message.textContent = "Please Enter Your Feedback Before Submitting.";
36                 message.style.color = "red";
37             } else {
38                 message.textContent = "Thank you! Your Feedback Has Been Submitted.";
39                 message.style.color = "green";
40                 textarea.value = "";
41             }
42         });
43     </script>
44 </body>
45 </html>

```

Output File: index.html

Feedback Form

Write your Feedback Here....

Submit Feedback

Feedback Form

Write your Feedback Here....

Submit Feedback

Please Enter Your Feedback Before Submitting.

Feedback Form

Excellent

Submit Feedback

Feedback Form

Write your Feedback Here....

Submit Feedback

Thank you! Your Feedback Has Been Submitted.

Explanation:

This code creates a simple **Feedback Form** where the user can type feedback and click **Submit Feedback**. JavaScript checks if the textarea is empty before submitting. If no feedback is entered, it shows a **red error message** asking the user to enter feedback. If feedback is provided, it displays a **green success message** and clears the textbox.

Problem 2

Problem Statement: Theme Toggle Application (Level-1)

Scenario:

A small blog website wants to allow users to switch between Light Mode and Dark Mode interactively.

z'' Requirements

- Create a toggle button to switch themes.
- Change background and text color dynamically.
- Store theme preference in localStorage.

z\' Technical Constraints

- Use Vanilla JavaScript.
- Use addEventListener() for events.
- No CSS frameworks.

Expectations:

- Smooth theme switching.
- Proper usage of localStorage.
- Clean and modular JavaScript functions.

Learning Outcome •

Handling user interaction events.

- Managing browser storage.
- Source Code:

File Name: index.html

Output:

```
p2 > index.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Theme Toggle Application</title>
7  <style>
8  body {
9    font-family: Arial;
10   background-color: #white;
11   color: #black;
12   text-align: center;
13   padding-top: 100px;
14   transition: 0.3s;
15 }
16 .dark-mode {
17   background-color: #222;
18   color: #white;
19 }
20 button {
21   padding: 10px 20px;
22   margin-top: 20px;
23   cursor: pointer;
24 }
25 </style>
26 </head>
27 <body>
28 <div>
29 <h1>My Blog Website</h1>
30 <p>Welcome to My Blog Website.</p>
31 <p>You can Switch Between Light and Dark Mode.</p>
32 </div>
33 <button id="themeToggle">Toggle Theme</button>
34 <script>
35 const toggleBtn = document.getElementById("themeToggle");
36 const body = document.body;
37 // Load saved theme
38 const savedTheme = localStorage.getItem("theme");
39 if (savedTheme === "dark") {
40   body.classList.add("dark-mode");
41 }
42 // Toggle theme
43 toggleBtn.addEventListener("click", () => {
44   body.classList.toggle("dark-mode");
45   if (body.classList.contains("dark-mode")) {
46     localStorage.setItem("theme", "dark");
47   } else {
48     localStorage.setItem("theme", "light");
49   }
50 });
51 </script>
52 </body>
53 </html>
```

Output File: index.html

My Blog Website

Welcome to My Blog Website.

You can Switch Between Light and Dark Mode.

Toggle Theme

My Blog Website

Welcome to My Blog Website.

You can Switch Between Light and Dark Mode.

Toggle Theme

Explanation:

This code creates a simple **Theme Toggle** webpage where the user can switch between **Light mode and Dark mode** by clicking the **Toggle Theme** button. JavaScript adds or removes the **dark-mode class** to change the background and text color, and it also saves the selected theme in **localStorage** so the same theme stays when the page is refreshed.

Problem 3

Problem Statement: Dynamic To-Do List (Level-2)

Scenario:

A productivity application requires an interactive To-Do list where users can add, delete, and mark tasks as complete without refreshing the page.

z'' Requirements

- Add new tasks dynamically.
- Delete tasks on button click.
- Mark tasks as completed.
- Use event delegation for better performance.

z\'' Technical Constraints •

Must use DOM manipulation.

- Event delegation required.
- Code should follow modular structure.

Expectations:

- Fully functional interactive UI.
- Organized and reusable functions.

Learning Outcome

- Advanced DOM event handling.
- Event delegation concept.
- Understanding role of libraries in speeding up development.

Source Code:

File Name: index.html

```
p4 > index.html > html
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Mini Product Search App</title>
7 <style>
8 body {
9   font-family: Arial;
10  text-align: center;
11  margin-top: 50px;
12  background-color: lightcyan;
13 }
14 input {
15   padding: 8px;
16   width: 250px;
17   margin-bottom: 20px;
18 }
19 ul {
20   list-style: none;
21   padding: 0;
22 }
23 li {
24   background: #f2f2f2;
25   margin: 5px;
26   padding: 10px;
27 }
28 li:hover {
29   background: #ddd;
30   cursor: pointer;
31 }
32 </style>
33 </head>
34 <body>
35 <h2>Product Search</h2>
36 <input type="text" id="searchBox" placeholder="Search products...">
37 <ul id="productList"></ul>
38 <p id="noResult" style="display:none;">No Results Found</p>
39 <script>
40 const searchBox = document.getElementById("searchBox");
41 const productList = document.getElementById("productList");
42 const noResult = document.getElementById("noResult");
43 const products = [
44   "Laptop",
45   "Mobile Phone",
46   "Headphones",
47   "Keyboard",
48   "Mouse",
49   "Smart Watch",
50   "Tablet"
51 ];
52 const displayProducts = (items) => {
53   productList.innerHTML = "";
54   if (items.length === 0) {
```

```
54   if (items.length === 0) {
55     noResult.style.display = "block";
56     return;
57   }
58   noResult.style.display = "none";
59   items.forEach(product => {
60     const li = document.createElement("li");
61     li.textContent = product;
62     productList.appendChild(li);
63   });
64 };
65 const filterProducts = () => {
66   const searchText = searchBox.value.toLowerCase();
67   const filtered = products.filter(product => product.toLowerCase().includes(searchText));
68   displayProducts(filtered);
69 };
70 searchBox.addEventListener("input", filterProducts);
71 productList.addEventListener("click", (event) => {
72   if (event.target.tagName === "LI") {
73     alert(`You selected: ${event.target.textContent}`);
74   }
75 });
76 displayProducts(products);
77 </script>
78 </body>
79 </html>
```

Output:

Output File: index.html

To-Do List

To-Do List

To-Do List

I need to do Hands-On Work

To-Do List

~~I need to do Hands-On Work~~

Explanation:

This code creates a simple **To-Do List web app** where users can type a task and click **AddTask** to display it in the list. When a task is added, it appears with a **Delete button**, and clicking the task marks it as **completed (line-through)**. Users can also **remove tasks** by clicking the delete button.

Problem 4

Problem Statement: Mini Product Search App (Level-2)

Scenario:

An e-commerce company needs a mini product search application that filters products dynamically as users type in the search box.

🔴 Requirements

- Display product list from a predefined array.
- Filter products dynamically using input event.
- Display “No Results Found” when applicable.

🔵 Technical Constraints •

Must use DOM manipulation.

- Event delegation required.
- Code should follow modular structure.

Expectations:

- Real-time filtering.
- Proper state management.

🎯 Learning Outcome

- State management basics.

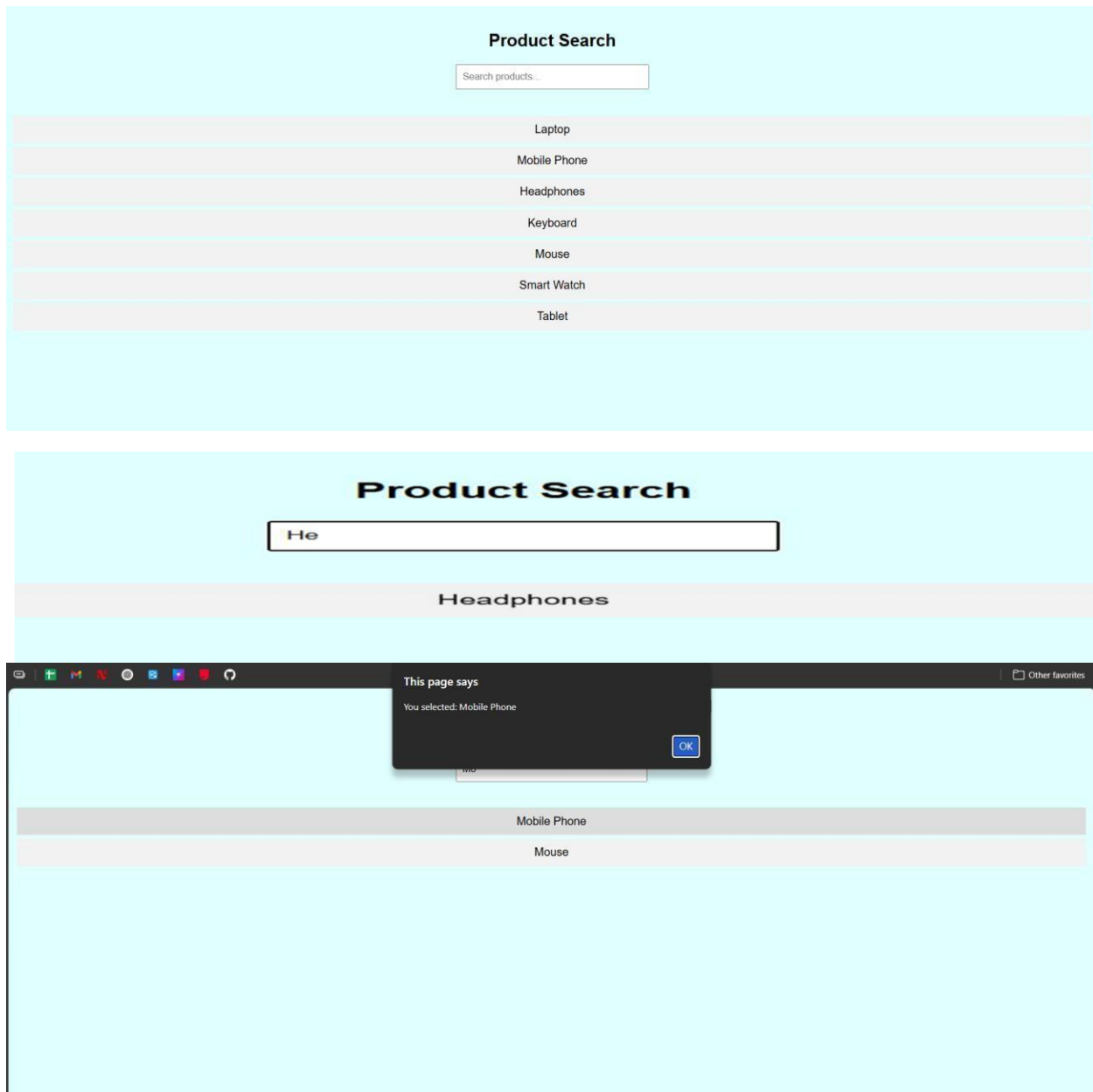
Source Code:

File Name: index.html

```
p4 > index.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Mini Product Search App</title>
7  </head>
8  <body>
9    <input type="text" id="searchBox" placeholder="Search products...">
10   <ul id="productList"></ul>
11   <p id="noResult" style="display:none;">No Results Found</p>
12   <script>
13     const searchBox = document.getElementById("searchBox");
14     const productList = document.getElementById("productList");
15     const noResult = document.getElementById("noResult");
16     const products = [
17       "Laptop",
18       "Mobile Phone",
19       "Headphones",
20       "Keyboard",
21       "Mouse",
22       "Smart Watch",
23       "Tablet"
24     ];
25     const displayProducts = (items) => {
26       productList.innerHTML = "";
27       if (items.length === 0) {
28         noResult.style.display = "block";
29         return;
30       }
31       noResult.style.display = "none";
32       items.forEach(product => {
33         const li = document.createElement("li");
34         li.textContent = product;
35         productList.appendChild(li);
36       });
37     };
38     const filterProducts = () => {
39       const searchText = searchBox.value.toLowerCase();
40       const filtered = products.filter(product => product.toLowerCase().includes(searchText));
41       displayProducts(filtered);
42     };
43     searchBox.addEventListener("input", filterProducts);
44     productList.addEventListener("click", (event) => {
45       if (event.target.tagName === "LI") {
46         alert(`You selected: ${event.target.textContent}`);
47       }
48     });
49     displayProducts(products);
50   </script>
51 </body>
52 </html>
```

Output:

Output File: index.



Explanation:

This code creates a simple **Product Search app** where a list of products is shown on the page. When the user types something in the search box, JavaScript **filters the products in real time** and only matching items are displayed. If a user clicks on any product, an **alert message shows the selected product name**, and if nothing matches the search, it displays **"No Results Found."**